📘 Java Collection Framework (JCF) – Complete Notes (Beginner to Advanced)

1. Introduction

What is the Java Collection Framework?

The **Java Collection Framework (JCF)** is a set of **interfaces, classes, and algorithms** provided by Java to efficiently store, organize, manipulate, and retrieve groups of objects.

Instead of creating your own data structures from scratch, Java provides ready-made implementations such as **ArrayList, HashSet, HashMap, LinkedList**, and many more.

It is available in the **java.util** package.

Why Do We Need Collections?

Imagine you want to store the names of 1,000 students.

Without Collections

- String student1 = "John";
- String student2 = "David";
- String student3 = "Emma";

...

This approach is not practical.

With Collections

- ArrayList<String> students = new ArrayList<>();
-
- students.add("John");
- students.add("David");

```
students.add("Emma");
```

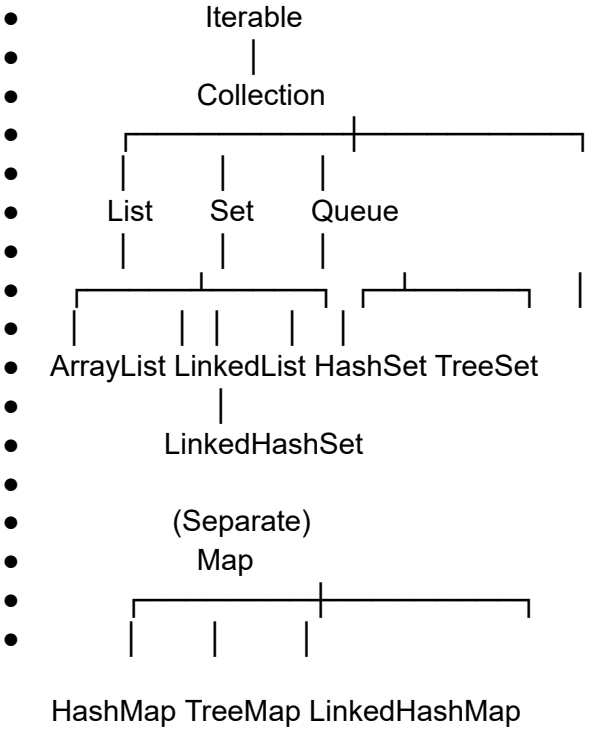
Now you can store millions of objects inside a single collection.

Advantages of Java Collection Framework

- Reduces programming effort
- Provides reusable data structures
- Improves application performance
- Makes code easier to maintain
- Supports dynamic memory allocation
- Includes built-in searching and sorting algorithms
- Provides thread-safe collections for concurrent applications

---

2. Collection Framework Architecture



Note: `Map` is **not** a child of the `Collection` interface, but it is an important part of the Java Collection Framework.

---

3. Main Components of JCF

The Java Collection Framework consists of three major components.

A. Interfaces

Interfaces define the rules that collection classes must follow.

Examples:

- List
- Set
- Queue
- Map

B. Classes

Classes implement the interfaces.

Examples:

- ArrayList
- LinkedList
- HashSet
- TreeSet
- HashMap
- TreeMap

C. Algorithms

Java provides many ready-made algorithms through the `Collections` utility class.

Examples:

- Collections.sort(list);
-
- Collections.reverse(list);
-
- Collections.shuffle(list);

-

Collections.binarySearch(list, value);

---

4. Iterable Interface

`Iterable` is the root interface of the Collection hierarchy.

It allows collections to be traversed using the enhanced for-each loop.

Example:

- ArrayList<String> list = new ArrayList<>();
-
- list.add("Java");
- list.add("Python");
-
- for(String language : list)
- {
- System.out.println(language);

}

Output

- Java

Python

---

5. Collection Interface

The `Collection` interface is the parent interface of List, Set, and Queue.

Common methods include:

- add()
-
- remove()
-
- contains()
-

- size()
-
- isEmpty()
-
- clear()
-

iterator()

Example

- Collection<Integer> numbers = new ArrayList<>();
-
- numbers.add(10);
- numbers.add(20);
-

System.out.println(numbers.size());

Output

2

---

6. List Interface

Definition

A List is an ordered collection that allows duplicate elements.

Characteristics

- Maintains insertion order
- Allows duplicates
- Supports index-based access

Example

[10, 20, 30, 20]

Common Methods

- add()

-
- remove()
-
- get()
-
- set()
-
- contains()
-
- indexOf()
-

lastIndexOf()

Example

- List<String> languages = new ArrayList<>();
-
- languages.add("Java");
- languages.add("Python");
- languages.add("Java");
-

System.out.println(languages);

Output

[Java, Python, Java]

---

7. ArrayList

Internal Working

ArrayList is implemented using a **dynamic array**.

When the array becomes full, Java automatically creates a larger array and copies all elements into it.

Example

- Index

-
- 0 1 2 3
-

A B C D

Advantages

- Fast random access
- Less memory overhead
- Easy to use

Disadvantages

- Slow insertion in the middle
- Slow deletion in the middle

Time Complexity

Operation	Complexity
get()	O(1)
add()	O(1) (Amortized)
remove()	O(n)

---

8. LinkedList

Internal Working

LinkedList uses a **Doubly Linked List**.

A ⇄ B ⇄ C ⇄ D

Each node stores:

- Data
- Address of previous node
- Address of next node

Advantages

- Fast insertion
- Fast deletion

Disadvantages

- Slow random access
- More memory usage

Time Complexity

Operation	Complexity
Search	O(n)
Insert	O(1)
Delete	O(1)

---

9. Vector

Vector is similar to ArrayList but is synchronized.

Features

- Thread-safe
- Slower than ArrayList
- Legacy class

Use Vector only when synchronization is required.

---

10. Stack

Stack follows the **LIFO (Last In, First Out)** principle.

Example

- Push 10
-
- Push 20
-
- Push 30
-
- Stack
-
- 30
- 20

10

Methods

- push()
-
- pop()
-

peek()

---

11. Set Interface

A Set stores only unique elements.

Characteristics

- No duplicates
- No index
- Mostly unordered

Example

Input

- 10
-
- 20
-
- 20
-

30

Output

- 10
-
- 20
-

30

---

12. HashSet

HashSet uses a Hash Table internally.

Features

- No duplicates
- Fastest Set implementation
- Does not maintain insertion order

Example

- HashSet<Integer> set = new HashSet<>();
-
- set.add(30);
- set.add(20);
- set.add(10);
-

System.out.println(set);

Possible Output

[20, 10, 30]

---

13. LinkedHashSet

Features

- Maintains insertion order
- No duplicates

Example

- 30
-
- 20
-

  10

Output remains the same.

---

14. TreeSet

TreeSet stores elements in sorted order.

Internally uses a **Red-Black Tree**.

Example

Input

- 50
-
- 10
-
- 40
-

  20

Output

- 10
-
- 20

-
- 40
-

50

Time Complexity

Operation	Complexity
add()	O(log n)
remove()	O(log n)
contains()	O(log n)

---

15. Queue Interface

Queue follows the **FIFO (First In, First Out)** principle.

Real-life example:

People standing in a ticket queue.

Methods

- offer()
-
- poll()
-

peek()

Example

- Queue<Integer> queue = new LinkedList<>();
-
- queue.offer(10);
- queue.offer(20);
- queue.offer(30);
-

System.out.println(queue.poll());

Output

10

---

16. PriorityQueue

PriorityQueue removes elements based on priority instead of insertion order.

By default, Java implements it as a **Min Heap**.

Example

- PriorityQueue<Integer> pq = new PriorityQueue<>();
-
- pq.add(30);
- pq.add(10);
- pq.add(20);
-

System.out.println(pq.poll());

Output

10

---

17. Map Interface

A Map stores data in **Key–Value pairs**.

Example

- 101 → John
-

- 102 → David
-

103 → Emma

Characteristics

- Keys must be unique
- Values can be duplicated

---

18. HashMap

Features

- Fastest Map implementation
- Does not maintain order
- Allows one null key and multiple null values

Example

- HashMap<Integer, String> map = new HashMap<>();
-
- map.put(101, "John");
- map.put(102, "David");
-

System.out.println(map);

---

19. LinkedHashMap

Features

- Maintains insertion order
- Faster iteration than HashMap in some scenarios

---

20. TreeMap

TreeMap stores keys in sorted order.

Internally uses a **Red-Black Tree**.

Time Complexity

Operation	Complexity
put()	O(log n)
get()	O(log n)
remove()	O(log n)

---

21. Collections Utility Class

The `Collections` class provides useful algorithms for collections.

```
Collections.sort(list);
```

Sorts elements.

```
Collections.reverse(list);
```

Reverses the list.

```
Collections.shuffle(list);
```

Randomly shuffles elements.

```
Collections.binarySearch(list, value);
```

Performs binary search on a sorted list.

---

22. Time Complexity Cheat Sheet

Collection	Search	Insert	Delete	Ordered
ArrayList	O(1)*	O(1) (end)	O(n)	✅ Yes
LinkedList	O(n)	O(1)*	O(1)*	✅ Yes
HashSet	O(1)	O(1)	O(1)	❌ No
TreeSet	O(log n)	O(log n)	O(log n)	✅ Sorted
HashMap	O(1)	O(1)	O(1)	❌ No
TreeMap	O(log n)	O(log n)	O(log n)	✅ Sorted
PriorityQueue	O(n)	O(log n)	O(log n)	Priority

Note: Operations marked with `*` assume favorable conditions (e.g., access by index for `ArrayList`, or insertion/deletion at a known node position for `LinkedList`).

---

23. Which Collection Should You Choose?

Scenario	Recommended Collection
Frequent random access	ArrayList
Frequent insertion/deletion	LinkedList
Store unique elements	HashSet
Store unique sorted elements	TreeSet
Key–Value storage	HashMap
Sorted key–value storage	TreeMap
FIFO processing	Queue / LinkedList
Priority-based processing	PriorityQueue

Thread-safe map	ConcurrentHashMap
Thread-safe list	CopyOnWriteArrayList

---

24. Advanced Topics to Learn

After mastering the basics, continue with:

1. Generics
2. Iterator and ListIterator
3. Comparable vs Comparator
4. Fail-Fast vs Fail-Safe Iterators
5. Concurrent Collections
6. Immutable Collections (`List.of()`, `Set.of()`, `Map.of()`)
7. Streams API
8. Internal Working of ArrayList
9. Internal Working of HashMap
10. Internal Working of HashSet
11. Internal Working of ConcurrentHashMap
12. Performance Optimization and Best Practices